Python Básico

II. Objetos

Objetos compostos multidimensionais (matrizes)

NumPy e Pandas

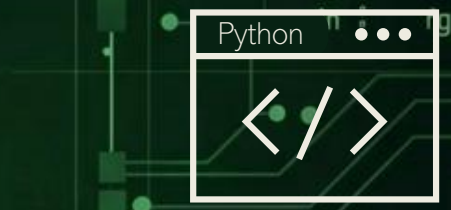
Guilherme Ferreira Silveira

gfsilveira@gmail.com

7 de abril de 2026

T7-E06

Conclusões Objetos Compostos Unidimensionais



- RAM
- Objetos Simples
- Objetos Compostos
- Valor e index
- nome_objeto_composto[index]
- Diferentes Objetos Compostos

Objetos Compostos -> características

Auto indexada
Imutável

```
1  nome_tupla = ("valor 1", "valor 2", "valor 3")
```

Auto indexada
Mutável

```
3  nome_lista = ["valor 1", "valor 2", "valor 3"]
```

Index manual
Mutável

```
5  nome_dicionario = {"index 1": "valor 1", "index 2": "valor 2",
```

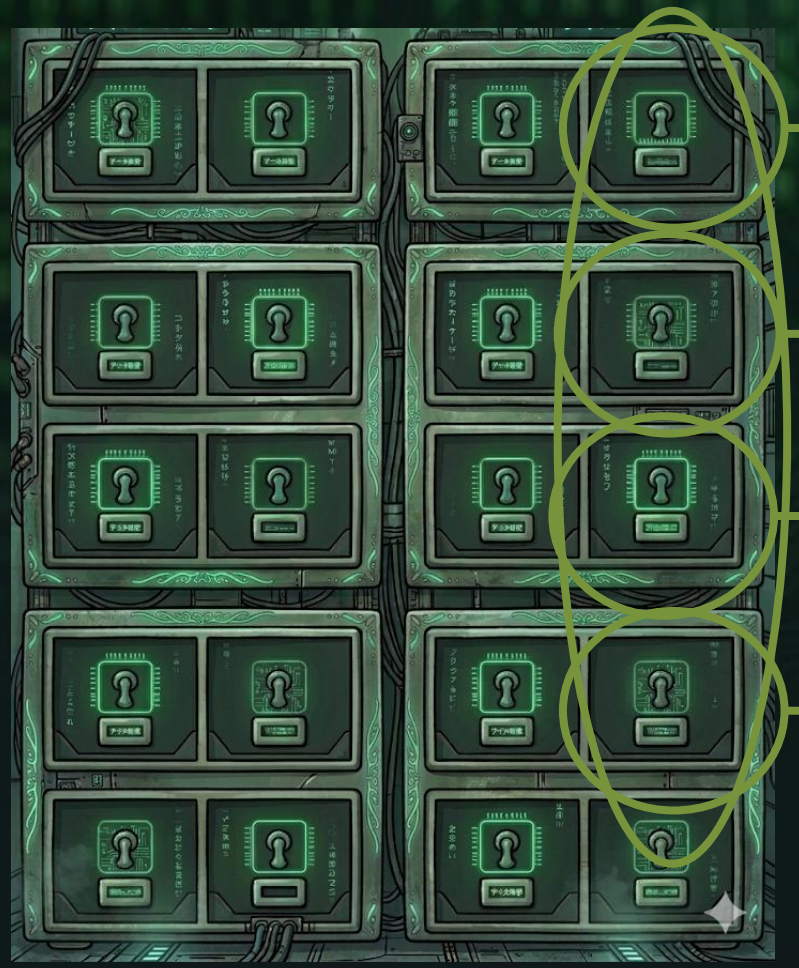
Não index
Imutável

```
7  nome_set = {"valor 1", "valor 2", "valor 3"}
```

Objetos Compostos Unidimensionais

RAM

lamps



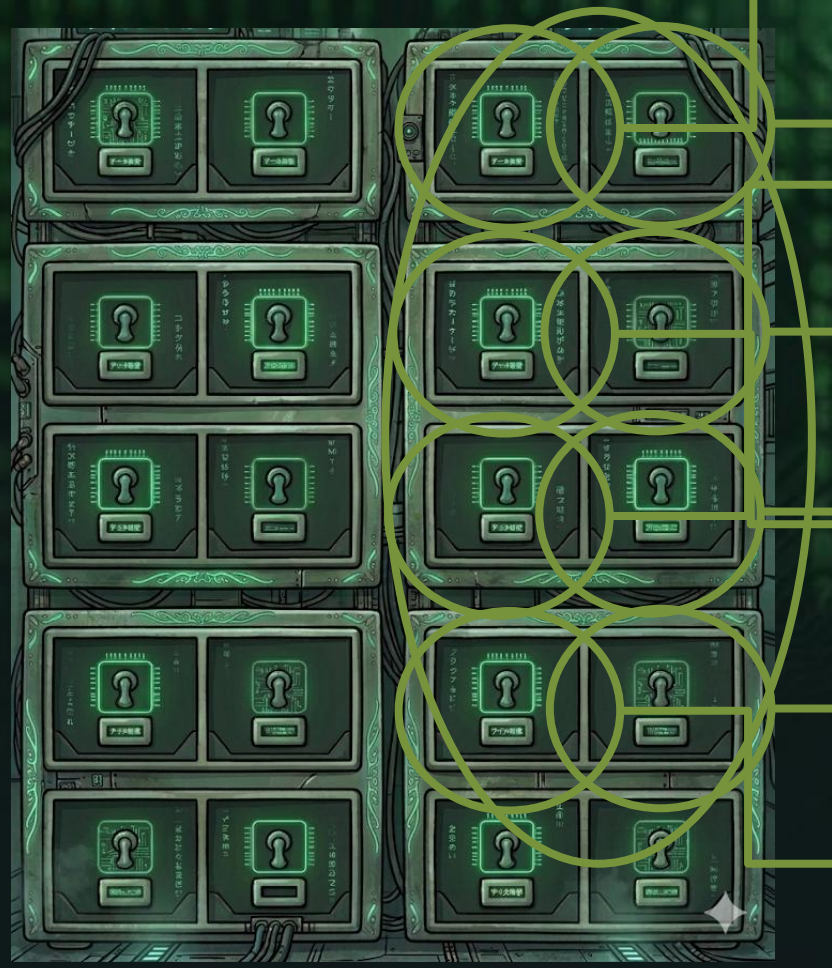
```python
lamps = [
    "ligada",
    "desligada",
    "desligada",
    "ligada",
]
```

[5] ✓ 0.0s

Objetos Compostos Multidimensional

RAM

matrix



```
matrix = [
    [1, 2],
    [3, 4],
    [5, 6],
    [7, 8]
]
matrix
```

[41] ✓ 0.0s

```
[[1, 2], [3, 4], [5, 6], [7, 8]]
```

Instanciando Objetos Compostos Multidimensionais

nome_objeto_simples *receber* valor

nome_objeto_composto *recebe* valor[index], valor[index], ... valor[index]

objeto_composto_mult recebe

objeto_composto_uni_1[1-index],

objeto_composto_uni_2[2-index],

objeto_composto_uni_3[3-index],

Instanciando Objetos Compostos Multidimensionais

objeto_composto_mult *recebe*

objeto_composto_uni_1[1-index],

objeto_composto_uni_2[2-index],

objeto_composto_uni_3[3-index],

```python
matrix = [
    [1, 2],
    [3, 4],
    [5, 6],
    [7, 8]
]
matrix
```

[41] ✓ 0.0s

```
[[1, 2], [3, 4], [5, 6], [7, 8]]
```

```python
locais = ['sala', 'cozinha', 'quarto', 'banheiro']
lampadas = ['ligada', 'desligada', 'quebrada', 'ligada']
num_lampdas = [2, 1, 1, 3]
testadas = [True, False, False, True]
metros_quadrados = [20.0, 15.2, 10.4, 5.9]

iluminação_1 = {
    'locais': locais,
    'lampadas': lampadas,
    'num_lampdas': num_lampdas,
    'testadas': testadas,
    'metros_quadrados': metros_quadrados
}
iluminação_1
```

[47] ✓ 0.0s

```
{'locais': ['sala', 'cozinha', 'quarto', 'banheiro'],
 'lampadas': ['ligada', 'desligada', 'quebrada', 'ligada'],
 'num_lampdas': [2, 1, 1, 3],
 'testadas': [True, False, False, True],
 'metros_quadrados': [20.0, 15.2, 10.4, 5.9]}
```

Instanciando Objetos Compostos Multidimensionais

```python
locais = ('sala', 'cozinha', 'quarto', 'banheiro')
lampadas = ['ligada', 'desligada', 'quebrada', 'ligada']
num_lampdas = (2, 1, 1, 3)
testadas = {
            locais[0]: True,
            locais[1]: False,
            locais[2]: False,
            locais[3]: True
}
metros_quadrados = {20.0, 15.2, 10.4, 5.9}

iluminação_2 = {
    'locais': locais,
    'lampadas': lampadas,
    'num_lampdas': num_lampdas,
    'testadas': testadas,
    'metros_quadrados': metros_quadrados
}
iluminação_2
```

[48] ✓ 0.0s

```
{'locais': ('sala', 'cozinha', 'quarto', 'banheiro'),
 'lampadas': ['ligada', 'desligada', 'quebrada', 'ligada'],
 'num_lampdas': (2, 1, 1, 3),
 'testadas': {'sala': True,
  'cozinha': False,
  'quarto': False,
  'banheiro': True},
 'metros_quadrados': {5.9, 10.4, 15.2, 20.0}}
```

```python
print(type(iluminação_2['locais']))
print(type(iluminação_2['lampadas']))
print(type(iluminação_2['num_lampdas']))
print(type(iluminação_2['testadas']))
print(type(iluminação_2['metros_quadrados']))
```

[49] ✓ 0.0s

```
<class 'tuple'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
```

Instanciando Objetos Compostos Multidimensionais

```python
matrix
```

[68] ✓ 0.0s

```
[[1, 2], [3, 4], [5, 6], [7, 8]]
```

```python
matrix[0][0] = "1"
matrix
```

[69] ✓ 0.0s

```
[['10', 2], [3, 4], [5, 6], [7, 8]]
```

```python
matrix[1] = "iluminação_2"
matrix
```

[79] ✓ 0.0s

```
[['1', 2], 'iluminação_2', [5, 6], [7, 8]]
```

```python
matrix.insert(4, ["iluminação_2", iluminação_2])
matrix
```

[82] ✓ 0.0s

```
[['1', 2],
 'iluminação_2',
 [5, 6],
 [7, 8],
 ['iluminação_2',
  {'locais': ('sala', 'cozinha', 'quarto', 'banheiro'),
   'lampadas': ['ligada', 'desligada', 'quebrada', 'ligada'],
   'num_lampdas': (2, 1, 1, 3),
   'testadas': {'sala': True,
    'cozinha': False,
    'quarto': False,
    'banheiro': True},
   'metros_quadrados': {5.9, 10.4, 15.2, 20.0}}]]
```

Módulos (Bibliotecas)

Python

Install Documentation Community About Us Contribute English

SciPy

Fundamental algorithms for scientific computing in Python

GET STARTED

SciPy 1.17.1 released! 2026-02-22

Fundamental algorithms

SciPy provides algorithms for optimization, integration, interpolation, eigenvalue problems, algebraic equations, differential equations, statistics and many other classes of problems.

Broadly applicable

The algorithms and data structures provided by SciPy are broadly applicable across domains.

Foundational

Extends NumPy providing additional tools for array computing and provides specialized data structures, such as sparse matrices and k-dimensional trees.

Performant

SciPy wraps highly-optimized implementations written in low-level languages like Fortran, C, and C++. Enjoy the flexibility of Python with the speed of compiled code.

Easy to use

SciPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

Open source

Distributed under a liberal BSD license, SciPy is developed and maintained publicly on GitHub by a vibrant, responsive, and diverse community.

Módulos (Bibliotecas)

Python

pandas

NumPy

```python
import pandas as pd
import numpy as np
```

[83] ✓ 0.6s

Numpy

```python
id = 'testadas'
array = np.array(iluminação_2['locais'])
array = np.append(array, iluminação_2['lampadas'])
array = np.append(array, iluminação_2['num_lampdas'])
array = np.append(array, [
                    iluminação_2[id][list(iluminação_2[id].keys())[0]],
                    iluminação_2[id][list(iluminação_2[id].keys())[1]],
                    iluminação_2[id][list(iluminação_2[id].keys())[2]],
                    iluminação_2[id][list(iluminação_2[id].keys())[3]]
                ]
)
array = array.reshape(4, -1)
array
```

[136] ✓ 0.0s

```
array([['sala', 'cozinha', 'quarto', 'banheiro'],
       ['ligada', 'desligada', 'quebrada', 'ligada'],
       ['2', '1', '1', '3'],
       ['True', 'False', 'False', 'True']], dtype='<U21')
```

```python
type(array)
```

[137] ✓ 0.0s

```
numpy.ndarray
```

```python
array.shape
```

[138] ✓ 0.0s

```
(4, 4)
```

```python
num_vetor = np.random.randint(0, 10, 9)
num_vetor
```

[149] ✓ 0.0s

```
array([2, 8, 5, 8, 7, 2, 6, 0, 9])
```

```python
num_matrix = np.random.randint(
                    0,
                    10,
                    size=(3, 3)
)
num_matrix
```

[142] ✓ 0.0s

```
array([[7, 2, 0],
       [0, 4, 2],
       [2, 0, 9]])
```

Numpy

```python
num_array = np.array((
    num_matrix,
    num_matrix,
    num_matrix,
))
num_array
```

[147] ✓ 0.0s

```
array([[[7, 2, 0],
        [0, 4, 2],
        [2, 0, 9]],

       [[7, 2, 0],
        [0, 4, 2],
        [2, 0, 9]],

       [[7, 2, 0],
        [0, 4, 2],
        [2, 0, 9]]])
```

```python
num_array.shape
```

[148] ✓ 0.0s

```
(3, 3, 3)
```

```python
num_matrix = np.random.randint(
                0,
                10,
                size=(3, 3)
)
num_matrix
```

[142] ✓ 0.0s

```
array([[7, 2, 0],
       [0, 4, 2],
       [2, 0, 9]])
```

Pandas

```python
df = pd.DataFrame(num_matrix)
df
```

[151] ✓ 0.0s

	0	1	2
0	7	2	0
1	0	4	2
2	2	0	9

```python
df = pd.read_csv('./data/data.csv')
df.head()
```

[155] ✓ 0.1s

	policyID	statecode	county	eq_site_limit	hu_site_limit	fl_site_limit	fr_site_limit	tiv_2011	tiv_2012	eq_site_deductible
0	119736	FL	CLAY COUNTY	498960.0	498960.00	498960.0	498960.0	498960.00	792148.90	0.0
1	448094	FL	CLAY COUNTY	1322376.3	1322376.30	1322376.3	1322376.3	1322376.30	1438163.57	0.0
2	206893	FL	CLAY COUNTY	190724.4	190724.40	190724.4	190724.4	190724.40	192476.78	0.0
3	333743	FL	CLAY COUNTY	0.0	79520.76	0.0	0.0	79520.76	86854.48	0.0
4	172534	FL	CLAY COUNTY	0.0	254281.50	0.0	254281.5	254281.50	246144.49	0.0

Conclusão

- Objetos Compostos Multidimensionais
- Instanciando
- nome_objeto_composto_mult[index]
- Valores, vetores, matrix
- Módulos
- NumPy
- Pandas

Vamos praticar?
LIMC-IA

Vamos exercitar?
LIMC-IA

Exercícios

1 – Um objeto composto multidimensional (matriz) pode receber objeto composto unidimensional (vetor)?

2 – No exercício 9 da última aula, foram instanciadas 5 listas. Instancie um objeto composto multidimensional da classe lista com estes cinco vetores. Recupere a matriz.

3 – Copie e converta o objeto instanciado no exercício 2 para um NumPy array. Recupere o array.

Exercícios

4 – Recupere todos os itens do terceiro gene.

5 – Verifique se todos os nomes dos genes tem ou não os caracteres 'hu'.

6 – Recupere do segundo e quarto genes, os dois primeiros caracteres dos nomes e os quantitativos de 'a' presentes nos nomes dos genes.

7 – Converta o objeto instanciado no exercício 10 da aula anterior para uma DataFrame Pandas. Nesta DataFrame, os index devem ser os nomes dos genes e cada coluna deve ter um nome que indique seu conteúdo. Recupere a DataFrame.

Exercícios

8 – Da DataFrame do exercício anterior recupere os números de caracteres e quantas vezes aparece o caractere ‘a’ de todos os genes.

9 – Da DataFrame do exercício anterior recupere se no nome aparecem os caracteres ‘hu’, para o primeiro e o terceiro genes.

10 – Importe o data.csv (dados de seguro na Flórida em 2012) para uma DataFrame. Armazene em um dicionário o número de entradas, uma lista com as classes das colunas e outra lista com o número de elementos de cada classe. Instancie ainda três listas, contendo a média, o desvio padrão e o terceiro quartil da primeira, quarta e nona colunas. Adicione ao dicionário anterior. Recupere o dicionário.

Python Básico

7 de abril de 2026